NEXT ENGINEER · _SHEERAT_

Internet of Things

BCS070

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT | **Semester:** 7 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 4th year, Departmental Elective-IV, 2025-26 onwards

UNIT 1 Introduction to IoT and Design Principles

UNIT 2 IoT Hardware and Platforms

UNIT 3 Network and Communication Aspects

UNIT 4 Programming the Arduino

UNIT 5 Challenges and Applications of IoT

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: architectures, layers, protocols and comparison tables.
3. **Solved examples**: the calculations and designs that come up in the paper (duty cycle and battery life, ADC conversion, sensor scaling, LEACH cluster-head probability, data aggregation savings, smart street light design).
4. **Code / key points to remember**: Arduino sketches you can reproduce in the exam.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (IoT vision, definition, conceptual framework, architecture, M2M, design principles) and Unit 2 (sensors, actuators, RFID, WSN, participatory sensing, embedded computing, Arduino, Raspberry Pi and other boards).
- **Afternoon (3 hours):** Unit 3 (wireless MAC issues, MAC and routing protocols, deployment, node discovery, data aggregation and dissemination) and Unit 4 (Arduino board, IDE, sketch structure, libraries, IoT programs).
- **Evening (2 hours):** Unit 5 (challenges, security, applications, smart street light design). Then the "Last-hour checklist".
- **In the exam:** draw a layered diagram for every architecture question, write protocol comparisons as tables, and for Arduino questions write the full sketch with `setup()` and `loop()`.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to IoT and Design Principles

✍️ 2-mark definitions

- **Internet of Things (IoT):** a global network of uniquely identifiable physical objects ("things") with embedded sensors, actuators, processing and communication, which collect and exchange data over the Internet and act on it with little or no human involvement.
- **IoT vision:** anything, anytime, anywhere connectivity, where everyday objects become smart, sense their context, communicate and cooperate to deliver services.
- **Thing:** any physical object with a unique identity (an address or tag) that can sense, act or communicate, for example a smart meter, a wearable or an RFID-tagged parcel.
- **M2M (machine-to-machine) communication:** direct data exchange between devices, usually over cellular or wired links, without human intervention; for example a vending machine reporting stock to a server.
- **Conceptual framework of IoT:** IoT = physical object + controller, sensor and actuator + Internet (Raj Kamal writes it as "Physical object + Controller, Sensor and Actuators + Internet = Internet of Things").
- **Gateway:** a device that connects local low-power networks (ZigBee, BLE) to the Internet, translating protocols and often filtering or aggregating data.
- **Edge / fog computing:** processing data close to where it is produced (at the device or gateway) instead of sending everything to the cloud, to cut latency and bandwidth.
- **Data enrichment:** adding context to raw sensor data, such as a timestamp, location, units or device identity, so it becomes useful information.
- **Data consolidation:** collecting data from many devices into one store, removing duplicates and combining it for analysis.
- **Interoperability:** the ability of devices from different vendors to work together, which needs common standards for protocols and data formats.

💡 Core concepts

Sources of IoT (where it came from)

- RFID and the Auto-ID Center at MIT (Kevin Ashton coined "Internet of Things" in 1999).
- Wireless sensor networks, embedded systems and microcontrollers getting cheap.
- IPv6, which gives enough addresses (2^128) for every device.
- Cheap cloud computing and big data analytics.
- Smartphones as universal user interfaces and gateways.
- Popular development boards (Arduino, Raspberry Pi) that made prototyping easy.

IoT characteristics

Characteristic	Meaning
Interconnectivity	Everything can be connected to the global network

Characteristic	Meaning
Things-related services	Privacy, semantic consistency between physical and virtual things
Heterogeneity	Different hardware platforms and networks working together
Dynamic changes	Device states (sleep, wake, connected) and counts change constantly
Enormous scale	Billions of devices, far more than people
Safety	Both personal data and physical safety must be protected
Connectivity	Network access and compatibility

IoT architecture: the layered views

Three-layer model (simplest, write this first):

Layer	Job	Examples
Perception (sensing) layer	Senses the physical world and identifies things	Sensors, RFID tags, cameras, actuators
Network layer	Moves data from things to processing	Wi-Fi, ZigBee, BLE, LoRaWAN, 4G/5G, gateways
Application layer	Delivers the service to the user	Smart home app, smart city dashboard

Five-layer model (adds two layers, scores better in a 7-mark answer): Perception, Transport (network), Processing (middleware: storage, analysis, cloud), Application, Business (business models, graphs, user privacy and profit).

The ITU-T reference model has four layers plus two cross-cutting capabilities: Device layer, Network layer, Service support and application support layer, Application layer, with Management capabilities and Security capabilities running across all four.

Architectural view (Raj Kamal's oneM2M-style view): devices and gateways in the device domain, the network domain (access and core network), and the applications domain (M2M applications and service capabilities). Data flows: device → gateway → network → server or cloud → application.

Technology behind IoT

Area	Technologies
Identification	RFID, NFC, barcodes, IPv6 addresses, EPC
Sensing	MEMS sensors, smart sensors
Hardware	Microcontrollers (ATmega, ARM Cortex-M), SoCs
Communication	IEEE 802.15.4, ZigBee, BLE, Wi-Fi, LoRa, NB-IoT, cellular
Protocols	6LoWPAN, RPL, CoAP, MQTT, XMPP, AMQP, HTTP/REST
Software and data	Cloud platforms, big data, analytics, ML
Security	Lightweight cryptography, TLS/DTLS, authentication

M2M vs IoT

Feature	M2M	IoT
Scope	Point-to-point between machines	Global network of things, people and services
Communication	Mostly cellular or wired, often proprietary	IP-based, open protocols
Data	Stored locally or at one server	Stored in the cloud and shared across apps
Integration	Vertical, one application	Horizontal, many applications share data
Scalability	Limited	Very large
Example	Remote meter reading	Smart city with meters, traffic and lighting linked

IoT examples: smart home (lights, thermostat), wearables (fitness band), smart agriculture (soil moisture irrigation), connected cars, smart grid metering, industrial IoT (predictive maintenance), healthcare (remote patient monitoring), logistics (RFID tracking).

Design principles for connected devices

1. **IoT/M2M layers and their standardization.** Raj Kamal's modified OSI view for IoT/M2M has five levels: 1 devices and gateway (physical cum data link), 2 network, 3 transport, 4 application support (data adaptation, data management), 5 application. Standards bodies: IETF (6LoWPAN, RPL, CoAP), IEEE (802.15.4, 802.11ah), ITU-T, ETSI M2M, oneM2M, OASIS (MQTT).
2. **Communication technologies.** Choose by range, data rate and power: NFC (cm), RFID (m), BLE and ZigBee (10–100 m), Wi-Fi (about 50 m, high rate), LoRaWAN and NB-IoT (km, low rate).
3. **Data enrichment, consolidation and device management at the gateway.** The gateway filters, aggregates, adds context, translates protocols and manages devices (provisioning, firmware updates, diagnostics).
4. **Ease of designing and affordability.** Use open hardware and open-source software, standard modules, low-cost microcontrollers, low power design and reusable cloud services, so products are cheap to build and scale.

Common IoT protocols (know the one-liners)

Protocol	Layer	Key point
IEEE 802.15.4	PHY/MAC	Low-rate WPAN, 250 kbps at 2.4 GHz, base of ZigBee and 6LoWPAN
6LoWPAN	Adaptation	Compresses IPv6 headers so IPv6 fits in 127-byte 802.15.4 frames
RPL	Network	IPv6 routing for low-power lossy networks, builds a DODAG
CoAP	Application	REST over UDP for constrained devices, GET/PUT/POST/DELETE, confirmable messages
MQTT	Application	Publish-subscribe over TCP through a broker, QoS 0/1/2, very small header (2 bytes)
XMPP	Application	XML messaging, presence, chat-like
AMQP	Application	Message queuing for enterprise, reliable delivery

🧮 Solved examples

Example 1: IPv6 address space. Why does IoT need IPv6? IPv4 has 2^32 ≈ 4.3 × 10^9 addresses, fewer than the number of connected devices expected (tens of billions). IPv6 has 2^128 ≈ 3.4 × 10^38 addresses, enough for every device to have a global address.

Example 2: MQTT topic matching. Topics: `home/kitchen/temp`, `home/hall/temp`, `home/kitchen/humidity`.

- Subscription `home/+/temp` (single-level wildcard) receives `home/kitchen/temp` and `home/hall/temp`.
- Subscription `home/kitchen/#` (multi-level wildcard) receives `home/kitchen/temp` and `home/kitchen/humidity`.
- Subscription `home/#` receives all three.

Example 3: Choosing a communication technology. A soil moisture sensor sends 20 bytes every 30 minutes from a farm 3 km from the nearest building, on battery for years. Wi-Fi is too short-range and power-hungry; BLE and ZigBee are too short-range without many relays; LoRaWAN (km range, very low power, low data rate) or NB-IoT (if cellular coverage exists) fits. Answer: LoRaWAN.

Key points to remember

- Kevin Ashton, 1999, coined "Internet of Things".
- Three-layer model: Perception, Network, Application. Five-layer adds Processing (middleware) and Business.
- M2M is vertical, point-to-point; IoT is horizontal, IP-based, cloud-centric.
- 6LoWPAN makes IPv6 work over 802.15.4; CoAP is REST over UDP; MQTT is pub-sub over TCP with a broker.
- Four design principles: layers and standards, communication technologies, data enrichment and consolidation at the gateway, ease of design and affordability.

⭐ Important questions

1. Define IoT. Explain the vision and characteristics of IoT with examples.
2. Explain the conceptual framework and the architectural view of IoT with a diagram.
3. Explain the technologies behind IoT and the sources of IoT.
4. What is M2M communication? Compare M2M and IoT.
5. Explain the design principles for connected devices: IoT/M2M layers, standardization, communication technologies, data enrichment and consolidation, ease of design and affordability.
6. Explain the three-layer and five-layer IoT architectures.

NEXT ENGINEER _sheerat_

UNIT 2 IoT Hardware and Platforms

✍️ 2-mark definitions

- **Sensor:** a device that detects a physical quantity (temperature, light, motion) and converts it into an electrical signal.
- **Transducer:** any device that converts energy from one form to another; a sensor is an input transducer and an actuator is an output transducer.
- **Analog sensor:** gives a continuous voltage or current proportional to the quantity, for example an LM35 temperature sensor or an LDR.
- **Digital sensor:** gives a discrete output (on/off or a digital value over I2C, SPI or 1-Wire), for example a DHT11 or DS18B20.
- **Actuator:** a device that converts an electrical signal into physical action, such as a motor, relay, solenoid, buzzer or LED.
- **RFID (Radio Frequency Identification):** identifying and tracking objects using tags that store an ID and are read wirelessly by a reader through radio waves, without line of sight.
- **Wireless sensor network (WSN):** a network of many small, battery-powered sensor nodes that sense the environment and cooperatively send data wirelessly to a sink (base station).
- **Participatory sensing:** sensing in which people use their own devices (mainly smartphones) to collect and share data, for example reporting potholes, noise or air quality.
- **Embedded system:** a computer system with a dedicated function inside a larger device, built around a microcontroller with limited memory and power.
- **Microcontroller:** a single chip with a CPU, RAM, flash, timers and I/O peripherals (ADC, UART, GPIO), for example ATmega328P.
- **ADC (analog-to-digital converter):** converts an analog voltage into a digital number; resolution is the number of bits.

💡 Core concepts

Sensor classification

Basis	Types
Output	Analog, digital
Power	Active (needs external power, e.g. strain gauge), passive (self-generating, e.g. thermocouple)
Quantity	Temperature, humidity, pressure, light, proximity, motion (PIR), gas, sound, acceleration, gyroscope
Contact	Contact (thermistor on skin), non-contact (IR thermometer, ultrasonic)

Sensor characteristics: range, sensitivity (output change per unit input), resolution (smallest detectable change), accuracy, precision (repeatability), linearity, response time, drift, hysteresis, offset.

Common sensors used in IoT

Sensor	Measures	Interface
LM35	Temperature, 10 mV/°C	Analog
DHT11 / DHT22	Temperature and humidity	Single-wire digital
LDR	Light (resistance falls in light)	Analog (voltage divider)
HC-SR04	Distance (ultrasonic, time of flight)	Digital trigger and echo
PIR (HC-SR501)	Human motion (infrared change)	Digital
MQ-2	Smoke, LPG gas	Analog
MPU-6050	Acceleration and rotation	I2C
Soil moisture probe	Water content of soil	Analog

Actuators

Actuator	Use
DC motor	Continuous rotation (fans, wheels), speed by PWM
Servo motor	Precise angle 0–180°, by PWM pulse width
Stepper motor	Fixed steps, open-loop position control
Relay	Switches high-voltage AC loads from a low-voltage signal
Solenoid	Linear push or pull (door lock, valve)
Buzzer, LED, LCD	Alarms and display

RFID

- Components: tag (chip + antenna), reader (antenna + transceiver), middleware or back-end database.
- **Passive tags** have no battery and are powered by the reader's field (short range, cheap). **Active tags** have a battery and transmit on their own (long range, about 100 m, costly). **Semi-passive** tags have a battery for the chip but use backscatter to reply.
- Frequency bands: LF 125–134 kHz (about 10 cm, animal tags), HF 13.56 MHz (about 1 m, NFC, smart cards), UHF 860–960 MHz (several metres, supply chain), microwave 2.45 GHz.
- Working: the reader sends a signal, the tag is energized and replies with its ID (backscatter), the reader passes the ID to middleware, which looks it up in a database.
- Applications: supply chain and inventory, toll collection (FASTag), access cards, library books, animal tracking, anti-theft.
- RFID vs barcode: no line of sight, reads many tags at once, read/write, harder to copy, but costs more.

Wireless sensor networks

- A sensor node has four units: sensing unit (sensor + ADC), processing unit (microcontroller + memory), transceiver (radio), power unit (battery, sometimes energy harvesting).
- Topologies: star, tree, mesh, cluster-based.
- Constraints: limited energy, memory and processing, unreliable radio links, large scale, self-organization needed.

- Applications: environment monitoring, forest fire detection, battlefield surveillance, structural health monitoring, precision agriculture.

WSN vs traditional network

WSN	Traditional network
Data-centric (what data, not which node)	Address-centric
Energy is the main constraint	Power is usually not a constraint
Many-to-one traffic to a sink	Any-to-any traffic
Nodes are cheap and can fail	Nodes are reliable
In-network processing (aggregation)	Processing at end hosts

Participatory sensing: people carry the sensors (phones with GPS, camera, microphone, accelerometer). Examples: Waze traffic reports, noise maps, air-quality crowdsourcing, pothole detection. Issues: privacy of location data, data quality and trust, incentives to take part, battery drain.

Embedded computing basics

- Microprocessor vs microcontroller: a microprocessor has only the CPU (memory and I/O are external, used in PCs); a microcontroller has CPU, memory and I/O on one chip (used in embedded systems, cheap, low power).
- Memory: flash (program), SRAM (variables), EEPROM (data that must survive power off).
- Interfaces: GPIO, UART (serial), I2C (two wires SDA and SCL, many devices by address), SPI (four wires, fast, full duplex), PWM (simulated analog output), ADC.
- Real-time requirements, power management (sleep modes), watchdog timer.

IoT development boards

Board	Processor	Key features	Best for
Arduino Uno	ATmega328P, 8-bit, 16 MHz	32 KB flash, 2 KB SRAM, 14 digital I/O (6 PWM), 6 analog inputs, no OS	Simple sensing and control
Netduino	ARM Cortex-M, 32-bit	Runs .NET Micro Framework, programmed in C#	.NET developers
Raspberry Pi 4	ARM Cortex-A72, 64-bit quad-core 1.5 GHz	2–8 GB RAM, Linux, Wi-Fi, BLE, Ethernet, HDMI, 40-pin GPIO, no analog input	Gateways, cameras, heavy processing
BeagleBone Black	ARM Cortex-A8, 1 GHz	512 MB RAM, Linux, 4 GB eMMC, 2 PRUs for real-time, 7 analog inputs	Industrial control
Intel Galileo	Intel Quark X1000, x86, 400 MHz	Arduino-compatible headers, Linux, Ethernet	x86 IoT prototyping
ARM Cortex-M boards (STM32, etc.)	ARM Cortex-M0/M3/M4	Low power, rich peripherals, RTOS support	Battery-powered products

Arduino vs Raspberry Pi (most asked)

Feature	Arduino Uno	Raspberry Pi
Type	Microcontroller board	Single-board computer
OS	None (runs one sketch)	Linux (Raspberry Pi OS)
Clock	16 MHz	1.5 GHz (Pi 4)
Memory	2 KB SRAM	2–8 GB RAM
Analog input	Yes (10-bit ADC)	No (needs external ADC)
Networking	Needs a shield or module	Built-in Wi-Fi, Ethernet, BLE
Power use	Very low	Higher
Language	C/C++ (Arduino)	Python, C, Java and others
Real-time control	Good	Weaker (OS scheduling)

🧮 Solved examples

Example 1: ADC conversion. Arduino Uno has a 10-bit ADC with 5 V reference. An LM35 gives 0.30 V. Find the ADC reading and temperature.

- Steps = 2^10 = 1024, so reading = 0.30 / 5 × 1024 = 61.44, stored as 61 (truncated).
- Back to voltage: 61 × 5 / 1024 = 0.2979 V. LM35 gives 10 mV/°C, so temperature = 0.2979 / 0.01 = 29.8 °C (the true value is 30 °C; the difference is quantization).
- Resolution = 5 / 1024 = 4.88 mV per step, which is about 0.49 °C per step.

Example 2: Ultrasonic distance. HC-SR04 echo pulse is 1166 µs. Speed of sound = 343 m/s.

- Distance = (time × speed) / 2 = (1166 × 10^-6 × 343) / 2 = 0.2 m = 20 cm. Divide by 2 because the sound goes to the object and back.
- Arduino shortcut: distance in cm = duration_µs / 58.3 ≈ 20 cm.

Example 3: Servo pulse. A servo uses a 50 Hz signal (period 20 ms). 1 ms pulse = 0°, 2 ms = 180°. For 90°, pulse = 1 + 90/180 × 1 = 1.5 ms; duty cycle = 1.5 / 20 = 7.5%.

Example 4: Voltage divider for an LDR. LDR in series with a fixed 10 kΩ resistor, 5 V supply, output across the 10 kΩ. In light the LDR is 1 kΩ: Vout = 5 × 10 / (10 + 1) = 4.55 V. In dark the LDR is 100 kΩ: Vout = 5 × 10 / (10 + 100) = 0.45 V. So brighter light gives higher voltage.

Key points to remember

- Sensor = input transducer, actuator = output transducer.
- Passive RFID has no battery; HF 13.56 MHz is NFC; UHF is used for supply chains.
- Sensor node = sensing + processing + transceiver + power.
- WSN is data-centric and energy-constrained.
- Arduino: microcontroller, no OS, analog inputs. Raspberry Pi: computer, Linux, no analog input.
- I2C has 2 wires, SPI has 4 wires, UART has TX and RX.

⭐ Important questions

1. What is a sensor? Classify sensors and explain their characteristics. Differentiate between sensors and actuators.
2. Explain RFID: components, types of tags, frequency bands, working and applications.
3. What is a wireless sensor network? Explain the architecture of a sensor node and the applications of WSN.
4. What is participatory sensing? Explain its applications and challenges.
5. Explain embedded computing basics: microcontroller vs microprocessor, memory and interfaces.
6. Compare Arduino, Raspberry Pi, BeagleBone, Netduino and Intel Galileo.

NEXT ENGINEER
sheerat

UNIT 3 Network and Communication Aspects

✍️ 2-mark definitions

- **Medium access control (MAC):** the protocol that decides which node may use the shared wireless channel and when, to avoid collisions.
- **Hidden terminal problem:** two nodes A and C cannot hear each other but both reach B; they transmit at the same time and collide at B.
- **Exposed terminal problem:** a node B hears a transmission from A and wrongly holds back from sending to D, even though its transmission would not interfere at the receiver.
- **Duty cycle:** the fraction of time a node's radio is on (active time / total period); low duty cycle saves energy.
- **Idle listening:** keeping the radio on while waiting for packets that may never come; the largest energy waste in WSNs.
- **Overhearing:** receiving packets meant for other nodes, which wastes energy.
- **Routing protocol:** the method that finds paths from sensor nodes to the sink across multiple hops.
- **Node discovery:** the process by which a node learns which other nodes are its neighbours (within radio range).
- **Data aggregation:** combining data from several nodes inside the network (sum, average, max, removing duplicates) to reduce the number of transmissions.
- **Data dissemination:** spreading data or queries through the network, from the sink to nodes or from nodes to the sink.
- **Sink (base station):** the node that collects all data from the WSN and connects it to the outside world.

💡 Core concepts

Wireless medium access issues

- Half-duplex radios: a node cannot listen while it transmits, so collision detection (CSMA/CD) does not work; wireless uses collision avoidance (CSMA/CA).
- Hidden and exposed terminal problems.
- Energy waste from collisions (retransmission), idle listening, overhearing, control packet overhead.
- Time-varying, error-prone channels; fading and interference.
- Fairness, latency and throughput must be traded against energy.

RTS/CTS solves the hidden terminal problem: the sender sends Request To Send, the receiver replies Clear To Send. Every node that hears CTS (including hidden nodes) stays silent for the time given in it (the Network Allocation Vector).

MAC protocol survey

Class	How it works	Protocols	Pros and cons
Contention-based (random access)	Nodes compete using carrier sensing and backoff	CSMA/CA, IEEE 802.11 DCF, B-MAC, S-MAC, T-MAC	Simple, scalable; collisions and idle listening
Schedule-based (TDMA)	Each node gets fixed time slots	TDMA, LEACH's intra-cluster TDMA, TRAMA	No collisions, sleep outside own slot; needs time sync, less flexible
Hybrid	Mix of both	Z-MAC, IEEE 802.15.4 (CSMA/CA plus guaranteed time slots in beacon mode)	Adapts to traffic

- **S-MAC (Sensor MAC):** nodes follow a periodic listen/sleep schedule, neighbours form virtual clusters with the same schedule, RTS/CTS for collisions, message passing for long data. Saves energy but adds latency.
- **T-MAC (Timeout MAC):** like S-MAC but ends the active period early when no activity is heard for a timeout, adapting to traffic.
- **B-MAC (Berkeley MAC):** low-power listening: nodes wake briefly to sample the channel; senders send a long preamble so the receiver catches it.
- **IEEE 802.15.4 MAC:** beacon-enabled mode with superframes (contention access period with slotted CSMA/CA, then guaranteed time slots), or non-beacon mode with unslotted CSMA/CA.

Routing protocol survey

Class	Idea	Protocols
Flat (data-centric)	All nodes equal; queries name the data	Flooding, gossiping, SPIN, Directed Diffusion
Hierarchical (cluster-based)	Nodes form clusters, cluster heads aggregate and forward	LEACH, PEGASIS, TEEN, APTEEN
Location-based (geographic)	Use node positions to forward towards the destination	GPSR, GEAR, GAF
QoS-based	Meet delay or reliability targets	SPEED, SAR
IoT standard	IPv6 routing for low-power lossy networks	RPL

- **Flooding:** every node rebroadcasts each packet. Simple but causes implosion (duplicate copies), overlap (neighbours sense the same area) and resource blindness.
- **Gossiping:** forward to one random neighbour; avoids implosion but slow.
- **SPIN:** negotiation with ADV (advertise metadata), REQ (request) and DATA messages, so data is only sent to nodes that want it.
- **Directed Diffusion:** the sink broadcasts an interest; nodes set up gradients; matching data flows back along the best gradient (reinforced path).
- **LEACH (Low-Energy Adaptive Clustering Hierarchy):** rounds with a setup phase (nodes elect themselves cluster head with probability threshold T(n), clusters form) and a steady phase (members send to the cluster head in TDMA slots, the head aggregates and sends to the sink). Rotating the head role spreads the energy load.

- **PEGASIS:** nodes form a chain; each node sends only to its chain neighbour and one leader sends to the sink.
- **RPL:** builds a Destination-Oriented DAG (DODAG) rooted at the border router using an objective function (hop count or ETX); control messages DIO, DIS, DAO.

Sensor deployment and node discovery

- Deployment: deterministic (planned positions, for example in buildings) or random (dropped from aircraft over a forest). Goals: coverage (every point sensed by at least k nodes) and connectivity (every node can reach the sink). Rule: if communication range ≥ 2 × sensing range, full coverage implies connectivity.
- Node discovery: nodes broadcast HELLO (beacon) messages, build neighbour tables, and exchange them to learn 2-hop neighbours. Localization methods: GPS (costly), range-based (RSSI, time of arrival, time difference of arrival, angle of arrival) and range-free (centroid, DV-hop) using anchor nodes with known positions.

Data aggregation and dissemination

- Aggregation functions: sum, average, min, max, count, duplicate suppression, compression.
- Approaches: tree-based (aggregate up a spanning tree, TAG), cluster-based (cluster heads aggregate, LEACH), chain-based (PEGASIS), in-network query processing (TinyDB).
- Benefits: fewer transmissions, longer battery life, less congestion. Costs: delay, loss of raw detail, a corrupted aggregator can poison results.
- Dissemination: push (nodes send events proactively), pull (sink queries), push-pull hybrid; examples flooding, Directed Diffusion, SPIN, rumour routing, Trickle (code updates).

🧮 Solved examples

Example 1: Duty cycle and battery life. A node draws 20 mA when awake and 0.02 mA asleep. It is awake 50 ms every 5 s. Battery 2000 mAh.

- Duty cycle = 0.05 / 5 = 1%.
- Average current = 0.01 × 20 + 0.99 × 0.02 = 0.2 + 0.0198 = 0.2198 mA.
- Life = 2000 / 0.2198 = 9099 h ≈ 379 days (about 1 year). Always on: 2000 / 20 = 100 h ≈ 4 days.

Example 2: LEACH threshold. Desired cluster-head fraction P = 0.05, so a node can be head once every 1/P = 20 rounds. In round r = 3 (counting from 0), for a node that has not been head in the current 20-round cycle:

- $T(n) = P / (1 - P \times (r \bmod 1/P)) = 0.05 / (1 - 0.05 \times 3) = 0.05 / 0.85 = 0.0588$.
- The node picks a random number in [0, 1]; if it is below 0.0588 it becomes a cluster head. Nodes that have already been head in this cycle have T(n) = 0. In round 19, T(n) = 0.05 / (1 − 0.95) = 1, so every remaining node becomes head.

Example 3: Aggregation saving. A tree has a sink with 3 children, and each child has 3 leaf children (9 leaves, 12 sensor nodes in total). Each node produces one reading per round.

- Without aggregation: each leaf reading travels 2 hops (9 × 2 = 18) and each middle node reading travels 1 hop (3), total 21 transmissions.
- With aggregation (each middle node sends one average for itself and its 3 children): leaves send 9, middle nodes send 3, total 12 transmissions. Saving = 9 / 21 = 43%.

Example 4: Hidden terminal. A, B, C in a line, A reaches B, C reaches B, A and C out of range. A senses the channel free and sends to B; C also senses free and sends to B. Both packets collide at B. With RTS/CTS, B's CTS to A is heard by C, which defers.

Key points to remember

- Wireless uses CSMA/CA, not CSMA/CD; RTS/CTS fixes hidden terminals.
- Energy wasters: collisions, idle listening, overhearing, control overhead.
- S-MAC: periodic sleep; T-MAC: timeout; B-MAC: low-power listening with long preamble.
- LEACH: rotating cluster heads, $T(n) = P / (1 - P (r \bmod 1/P))$.
- Flooding problems: implosion, overlap, resource blindness.
- RPL builds a DODAG with DIO, DIS, DAO.

⭐ Important questions

1. Explain the issues in wireless medium access. Explain hidden and exposed terminal problems and how RTS/CTS helps.
2. Give a survey of MAC protocols for WSN: contention, schedule-based and hybrid, with S-MAC in detail.
3. Give a survey of routing protocols for WSN. Explain flooding, SPIN, Directed Diffusion and LEACH.
4. Explain sensor deployment, coverage, connectivity and node discovery.
5. What is data aggregation? Explain its approaches and benefits. Explain data dissemination.

UNIT 4 Programming the Arduino

✍️ 2-mark definitions

- **Arduino:** an open-source electronics platform with easy-to-use hardware (microcontroller boards) and software (the Arduino IDE and language based on C/C++).
- **Sketch:** an Arduino program, saved with the `.ino` extension.
- **`setup()` :** runs once after power on or reset, used to set pin modes and start serial communication.
- **`loop()` :** runs repeatedly forever after `setup()` , containing the main logic.
- **Arduino IDE:** the development environment used to write, compile (verify) and upload sketches to the board through USB.
- **Bootloader:** a small program in the microcontroller's flash that lets new sketches be uploaded over serial without a hardware programmer.
- **PWM (Pulse Width Modulation):** rapidly switching a digital pin on and off; the duty cycle sets the average voltage, giving a pseudo-analog output (`analogWrite` , value 0–255).
- **Library:** a reusable collection of code for a sensor, module or task (for example `Servo.h` , `DHT.h` , `WiFi.h`), included with `#include` .
- **Shield:** an add-on board that plugs on top of the Arduino to add a feature (Ethernet, Wi-Fi, motor driver, GSM).
- **Emulator / simulator:** software that runs Arduino code on a virtual board, for example Tinkercad Circuits, Wokwi, Proteus or SimulIDE.

💡 Core concepts

Anatomy of the Arduino Uno board

Part	Detail
Microcontroller	ATmega328P, 16 MHz crystal
Digital I/O pins	14 (D0–D13), 6 with PWM (marked ~: 3, 5, 6, 9, 10, 11)
Analog input pins	6 (A0–A5), 10-bit ADC
Serial pins	D0 (RX), D1 (TX)
I2C pins	A4 (SDA), A5 (SCL)
SPI pins	D10 (SS), D11 (MOSI), D12 (MISO), D13 (SCK)
Built-in LED	Pin 13
Power	USB 5 V or barrel jack 7–12 V; pins 5V, 3.3V, GND, Vin
Memory	32 KB flash (0.5 KB bootloader), 2 KB SRAM, 1 KB EEPROM
Other	Reset button, USB-to-serial chip (ATmega16U2), ICSP header, voltage regulator

Arduino IDE workflow: write sketch → select board and port (Tools menu) → Verify (compile) → Upload → open Serial Monitor to see output. The IDE uses avr-gcc to compile and avrdude to upload.

Core functions

Function	Purpose
`pinMode(pin, INPUT / OUTPUT / INPUT_PULLUP)`	Sets pin direction
`digitalWrite(pin, HIGH / LOW)`	Sets a digital output
`digitalRead(pin)`	Reads HIGH or LOW
`analogRead(pin)`	Reads 0–1023 from A0–A5
`analogWrite(pin, 0-255)`	PWM output on ~ pins
`delay(ms)` , `millis()`	Wait; time since start in ms
`Serial.begin(9600)` , `Serial.println(x)`	Serial communication
`pulseIn(pin, HIGH)`	Measures pulse length in µs
`map(x, a, b, c, d)`	Rescales a value from one range to another
`attachInterrupt(digitalPinToInterrupt(2), isr, RISING)`	Runs a function on a pin change

Arduino libraries: built-in (Servo, Wire for I2C, SPI, EEPROM, SoftwareSerial, LiquidCrystal) and third-party installed through the Library Manager (DHT sensor library, PubSubClient for MQTT, Adafruit libraries). A library has a header (.h) and source (.cpp) and often example sketches.

Programming for IoT with Arduino: add connectivity with an ESP8266/ESP32 module or an Ethernet/Wi-Fi shield, then send data to the cloud using HTTP (ThingSpeak, REST APIs) or MQTT (PubSubClient). The ESP32 itself can be programmed from the Arduino IDE and has built-in Wi-Fi and BLE.

🧮 Solved examples

Example 1: PWM value. To run an LED at 40% brightness, analogWrite value = 0.40 × 255 = 102. Average voltage = 102 / 255 × 5 = 2.0 V.

Example 2: map(). A potentiometer reads 0–1023; to control a servo at 0–180°, angle = map(val, 0, 1023, 0, 180). For val = 512, angle = 512 × 180 / 1023 = 90.09, and integer maths gives 90.

Example 3: Blink timing without delay. Using `millis()` , the LED toggles when `millis() - last >= 1000` , so the program can do other work in between (non-blocking), unlike `delay(1000)` which freezes the loop.

Code

1. Blink the built-in LED

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

2. Read an LM35 and print the temperature

```
const int sensorPin = A0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int raw = analogRead(sensorPin);          // 0..1023
  float volts = raw * 5.0 / 1024.0;
  float tempC = volts * 100.0;              // LM35: 10 mV per degree C
  Serial.print("Temperature: ");
  Serial.print(tempC);
  Serial.println(" C");
  delay(1000);
}
```

3. Ultrasonic distance with an alarm

```
const int trig = 9, echo = 10, buzzer = 8;

void setup() {
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  pinMode(buzzer, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(trig, LOW);  delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long duration = pulseIn(echo, HIGH);      // microseconds
  float cm = duration * 0.0343 / 2;
  Serial.println(cm);
  digitalWrite(buzzer, cm < 20 ? HIGH : LOW);
  delay(200);
}
```

4. Servo controlled by a potentiometer (library use)

```
#include <Servo.h>
Servo s;

void setup() {
  s.attach(9);
}

void loop() {
  int val = analogRead(A0);
  int angle = map(val, 0, 1023, 0, 180);
  s.write(angle);
  delay(15);
}
```

5. IoT: publish DHT11 readings over MQTT (ESP32 in the Arduino IDE)

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

DHT dht(4, DHT11);
WiFiClient net;
PubSubClient mqtt(net);

void setup() {
  dht.begin();
  WiFi.begin("ssid", "password");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  mqtt.setServer("broker.hivemq.com", 1883);
}

void loop() {
  if (!mqtt.connected()) mqtt.connect("node-01");
  mqtt.loop();
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  String msg = "{\"t\":" + String(t) + ",\"h\":" + String(h) + "}";
  mqtt.publish("home/room1/climate", msg.c_str());
  delay(10000);
}
```

Key points to remember

- Every sketch has `setup()` (once) and `loop()` (forever).
- Uno: 14 digital pins (6 PWM), 6 analog pins, 10-bit ADC, PWM 0–255, ADC 0–1023.
- `analogRead` returns 0–1023; `analogWrite` takes 0–255.
- Libraries are added with `#include` and installed from the Library Manager.
- Emulators: Tinkercad, Wokwi, Proteus.
- Distance = duration × 0.0343 / 2 (cm).

⭐ Important questions

1. Explain the anatomy of the Arduino Uno board with a labelled diagram.
2. Explain the Arduino IDE and the structure of a sketch. Write a program to blink an LED.
3. What are Arduino libraries? How are they used? Write a program using the Servo library.
4. Write an Arduino program to read a temperature sensor and display the value on the serial monitor.
5. Explain how Arduino is used for IoT. Write a program to send sensor data to the cloud.
6. What is an Arduino emulator? Name some and give their use.

NEXT ENGINEER
sheerat

UNIT 5 Challenges and Applications of IoT

✍️ 2-mark definitions

- **Smart metering:** electricity, water or gas meters that measure usage in real time and send it automatically to the utility, enabling remote reading, dynamic pricing and outage detection (Advanced Metering Infrastructure).
- **E-health (IoT in healthcare):** using wearable and implanted sensors to monitor patients remotely and send vital signs to doctors.
- **City automation (smart city):** using IoT to manage traffic, lighting, waste, parking, water and pollution in a city.
- **Home automation:** controlling lights, fans, appliances, locks and security in a home through sensors, a hub and a phone app or voice assistant.
- **Smart card:** a plastic card with an embedded chip that stores and processes data, used for payments, metro travel and ID; contact or contactless (RFID/NFC).
- **Botnet (Mirai):** a network of infected IoT devices controlled by an attacker; Mirai (2016) used default passwords on cameras and routers to launch huge DDoS attacks.
- **Interoperability challenge:** devices from different vendors using different protocols cannot easily work together.

💡 Core concepts

Design and development challenges

- Heterogeneity of devices, protocols and data formats; lack of common standards.
- Scalability to billions of devices and huge data volumes.
- Energy: devices on batteries for years; need low-power design and energy harvesting.
- Limited processing and memory on devices.
- Reliability and fault tolerance in harsh environments.
- Real-time response and latency (fog and edge computing help).
- Device management: provisioning, remote firmware update, monitoring.
- Cost of devices and deployment.
- Data storage, analytics and turning data into decisions.

Security challenges

Issue	Detail
Weak authentication	Default or hard-coded passwords (Mirai)
Insecure communication	Unencrypted traffic, sniffing, man-in-the-middle
Insecure firmware	No secure boot, unsigned updates
Physical attacks	Devices are left unattended and can be tampered with

Issue	Detail
Constrained devices	Too weak for heavy cryptography
Privacy	Continuous personal data (location, health, habits)
DoS and botnets	Compromised devices used for attacks
Lifecycle	Devices rarely patched, old devices left exposed

Countermeasures: unique credentials, lightweight cryptography and DTLS/TLS, secure boot and signed OTA updates, network segmentation, intrusion detection at the gateway, data minimization and anonymization, standards such as the OWASP IoT Top 10.

Other challenges: legal and regulatory (data ownership, consent), ethical issues, spectrum availability, connectivity in rural areas, e-waste, skills shortage.

Applications

Domain	How IoT is used
Smart metering	Smart meters with communication module send readings to a head-end system via PLC, RF mesh or cellular; utility gets real-time load data, does remote connect/disconnect, time-of-use billing, theft and outage detection
E-health	Wearables (ECG, pulse oximeter, glucose) send data via a phone gateway to the cloud; doctors get alerts; elderly fall detection; hospital asset tracking with RFID
City automation	Smart parking sensors, adaptive traffic signals, smart street lights, waste-bin fill sensors, air-quality monitoring, CCTV analytics
Automotive	Connected cars (telematics, OBD-II), vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) communication, predictive maintenance, fleet tracking, toll by FASTag
Home automation	Smart lights, thermostats, door locks, cameras, smoke detectors, voice assistants; hubs using ZigBee, Z-Wave, Wi-Fi
Smart cards	Contactless payment (NFC), metro cards (RFID), access control, e-passports
Agriculture	Soil moisture irrigation, weather stations, livestock tracking
Industry	Predictive maintenance, asset tracking, process monitoring (IIoT)

Communicating data with hardware units, mobiles and tablets: the device sends data to a gateway or cloud (HTTP REST or MQTT); a mobile app subscribes or calls an API and shows dashboards; commands go back the same way to actuators. Local options: Bluetooth Low Energy directly between the phone and the device, or Wi-Fi on the same network. Cloud platforms: ThingSpeak, Blynk, AWS IoT, Azure IoT Hub, Firebase.

Designing smart street lights (frequent 7-mark question)

1. **Requirements:** light only when it is dark and when people or vehicles are near; dim otherwise; report faults; central monitoring.
2. **Hardware per pole:** LED lamp with a driver (PWM dimming), LDR (ambient light), PIR or microwave motion sensor, microcontroller (Arduino/ESP32), relay or MOSFET, communication module (ZigBee, LoRa or GSM/NB-IoT), current sensor for fault detection, optional solar panel and battery.

3. **Logic:** if daylight, lamp off. If dark and no motion, lamp at 30%. If dark and motion, lamp at 100% for a set time, then dim. If the lamp is commanded on but current is zero, raise a fault.
4. **Network:** poles form a mesh to a gateway (segment controller), which connects to a cloud server.
5. **Cloud and app:** dashboard with on/off status, energy use, faults on a map, schedules and remote override.
6. **Benefits:** 50–70% energy saving, faster fault repair, lower maintenance, data for city planning.

🧮 Solved examples

Example 1: Smart street light energy saving. 1000 lights of 100 W, 12 hours a night. Conventional: 1000 × 100 × 12 = 1,200 kWh per night. Smart: full power 3 hours and 30% for 9 hours gives 1000 × (100 × 3 + 30 × 9) = 1000 × 570 Wh = 570 kWh per night. Saving = 630 / 1200 = 52.5%.

Example 2: Smart meter data volume. 100,000 meters send a 200-byte reading every 15 minutes. Readings per day per meter = 96. Data per day = 100,000 × 96 × 200 B = 1.92 × 10^9 B ≈ 1.92 GB per day, about 700 GB a year, which is why aggregation and cloud storage matter.

Code

Smart street light logic (Arduino)

```
const int ldr = A0, pir = 2, lamp = 9;
const int DARK = 300;                 // below this the sky is dark
unsigned long lastMotion = 0;

void setup() {
  pinMode(pir, INPUT);
  pinMode(lamp, OUTPUT);
}

void loop() {
  int light = analogRead(ldr);
  if (digitalRead(pir) == HIGH) lastMotion = millis();

  if (light > DARK) {
    analogWrite(lamp, 0);             // daytime: off
  } else if (millis() - lastMotion < 30000) {
    analogWrite(lamp, 255);           // dark and recent motion: full
  } else {
    analogWrite(lamp, 77);            // dark, no motion: about 30%
  }
  delay(100);
}
```

Key points to remember

- Challenges: development (heterogeneity, scale, energy, cost), security (weak passwords, no encryption, no updates, privacy), other (legal, standards, e-waste).
- Mirai botnet 2016: default passwords, DDoS.
- Smart meters: AMI, remote reading, time-of-use billing.

- Smart street light: LDR + PIR + dimmable LED + mesh + cloud dashboard, about 50% saving.

⭐ Important questions

1. Explain the development and security challenges of IoT and their solutions.
2. Explain IoT applications in smart metering, e-health and city automation.
3. Explain IoT in the automotive sector and home automation.
4. What are smart cards? Explain their role in IoT.
5. How does an IoT device communicate data with mobiles and tablets? Explain with a diagram.
6. Design a smart street lighting system using IoT: components, block diagram, working and benefits.

Last-hour checklist

- ☐ IoT definition, vision, characteristics; Kevin Ashton 1999; conceptual framework.
- ☐ Three-layer and five-layer architectures; ITU-T model; technologies table.
- ☐ M2M vs IoT table; four design principles; MQTT vs CoAP; 6LoWPAN; IPv6 count.
- ☐ Sensor classification and characteristics; sensor vs actuator; common sensors table.
- ☐ RFID tags and frequencies; WSN node architecture; WSN vs traditional; participatory sensing.
- ☐ Microcontroller vs microprocessor; I2C, SPI, UART; Arduino vs Raspberry Pi table; other boards.
- ☐ ADC example (29.8 °C); ultrasonic (20 cm); servo pulse (1.5 ms).
- ☐ Wireless MAC issues; hidden and exposed terminals; RTS/CTS; S-MAC, T-MAC, B-MAC.
- ☐ Routing: flooding problems, SPIN, Directed Diffusion, LEACH formula, PEGASIS, RPL.
- ☐ Deployment, coverage, node discovery; aggregation (43% saving) and dissemination.
- ☐ Duty cycle battery life (about 1 year at 1%).
- ☐ Uno anatomy table; setup/loop; core functions; libraries; emulators; the five sketches.
- ☐ Challenges; Mirai; applications table; smart street light design and 52.5% saving.

Books for this subject

The AKTU syllabus copy we could access did not show the book list for this elective, so these are the standard textbooks that cover its units. Check your college's copy of the syllabus for the official list. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Internet of Things: Architecture and Design Principles	Raj Kamal	McGraw Hill Education	WorldCat
2	Internet of Things: A Hands-On Approach	Arshdeep Bahga, Vijay Madisetti	Universities Press / VPT	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
3	Fundamentals of Wireless Sensor Networks: Theory and Practice	Waltenegus Dargie, Christian Poellabauer	Wiley	WorldCat
4	Getting Started with the Internet of Things	Cuno Pfister	O'Reilly Media	WorldCat
5	Arduino Cookbook	Michael Margolis	O'Reilly Media	WorldCat

The official Arduino documentation at docs.arduino.cc is free and covers the board, IDE and libraries in Unit 4.

Syllabus source: AKTU B.Tech CS/CE and CSE 4th year syllabus, 2025-26 (copy hosted by an affiliated college, PDF), with the elective unit list cross-checked against aktunotes.in.

NEXT ENGINEER
sheerat